

Lookahead Information Passing in DuckDB

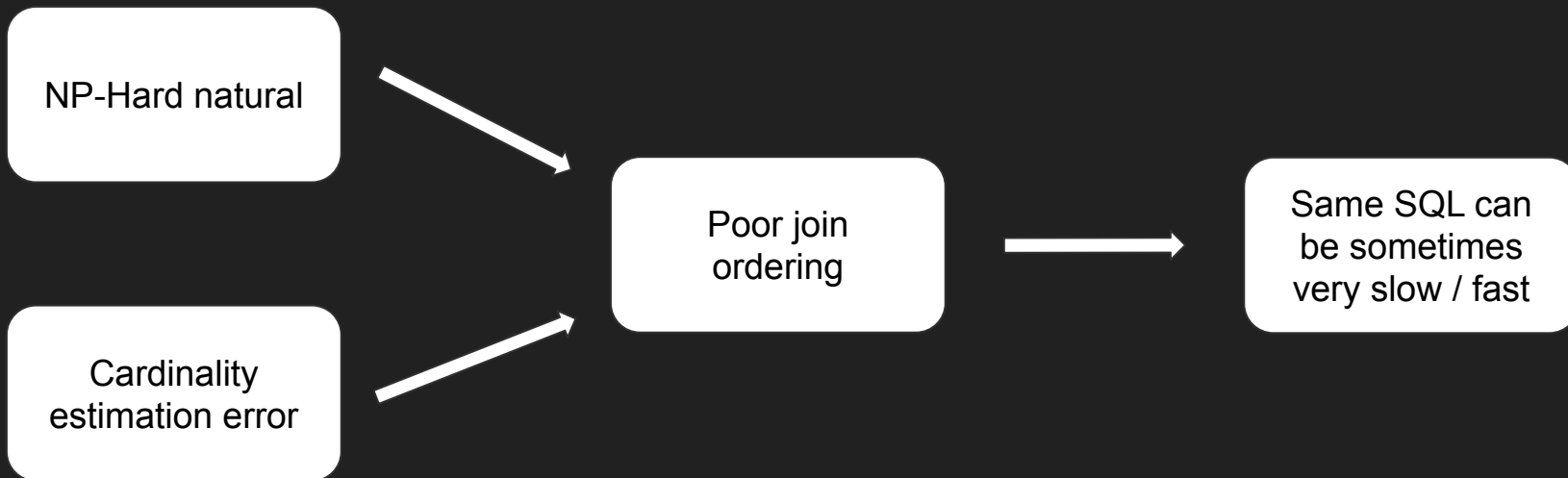
Overview

- Challenges
- Solution - Lookahead Information Passing
- Results
- Conclusion

Challenges

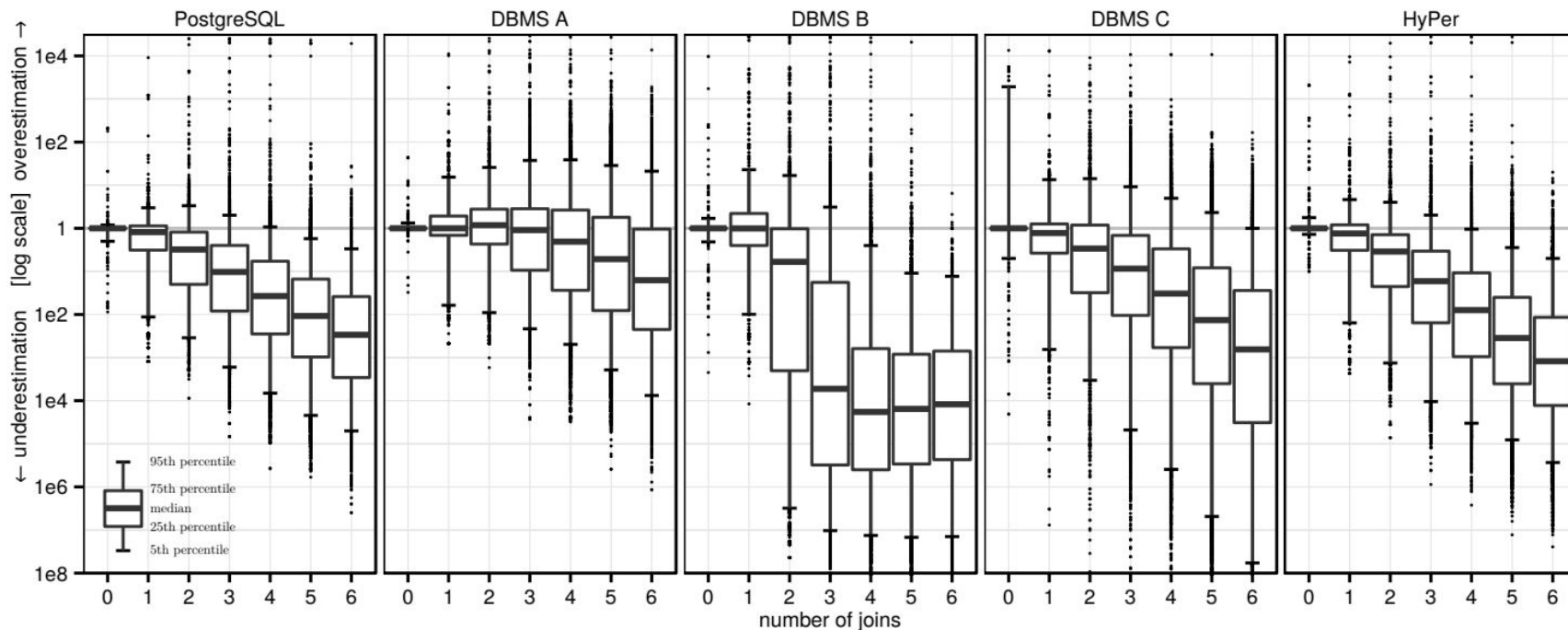
Challenges in Modern Analytic Systems

- Star schema is non-trivial
- But hard to optimize the join ordering
- If the optimizer pick a poor join order, the query can be very slow



Cardinality estimation errors grow exponentially over successive joins.

Optimizers Make Estimation Errors



Can we develop a query execution technique that mitigate
/ immune to poor plan selection?

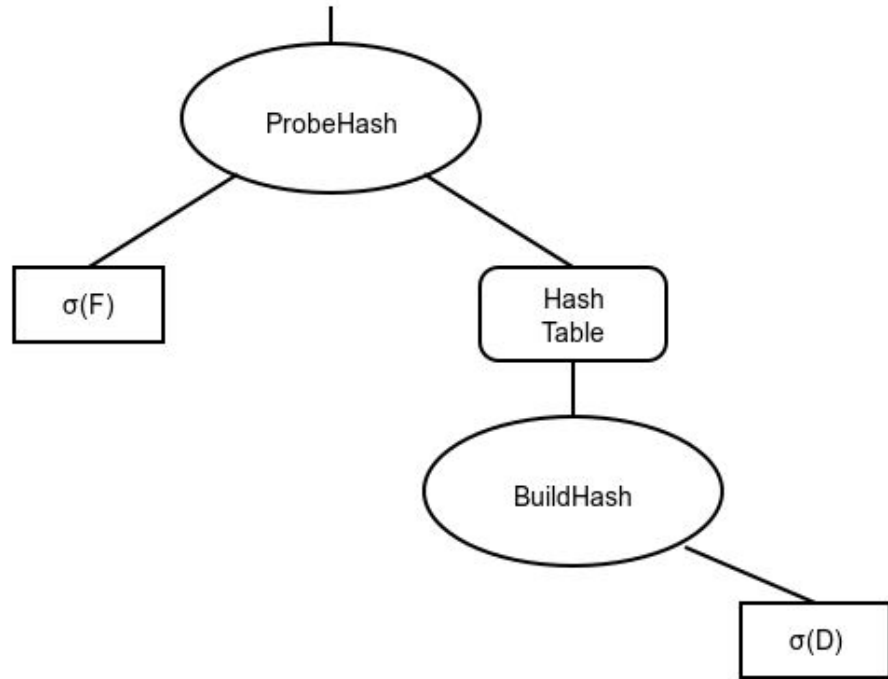
Solution - Lookahead Information Passing

J. Zhu, et al.: Looking Ahead Makes Query Plans Robust. Proc. VLDB Endow. (2017).

Lookahead Information Passing (LIP) : Scope

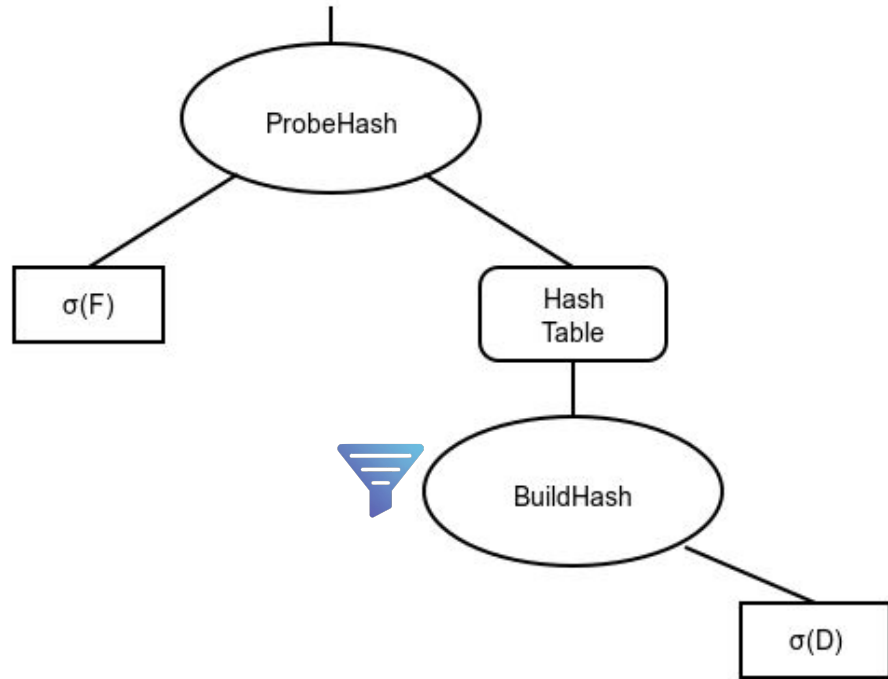
- Star Schema: 1 Fact table + n Dimension tables
- Select – Project – Join (SPJ) subqueries: Primarily inner joins
- Hash joins
- Join order selection: Left-deep join trees
- In-memory setting

Hash Join Optimization



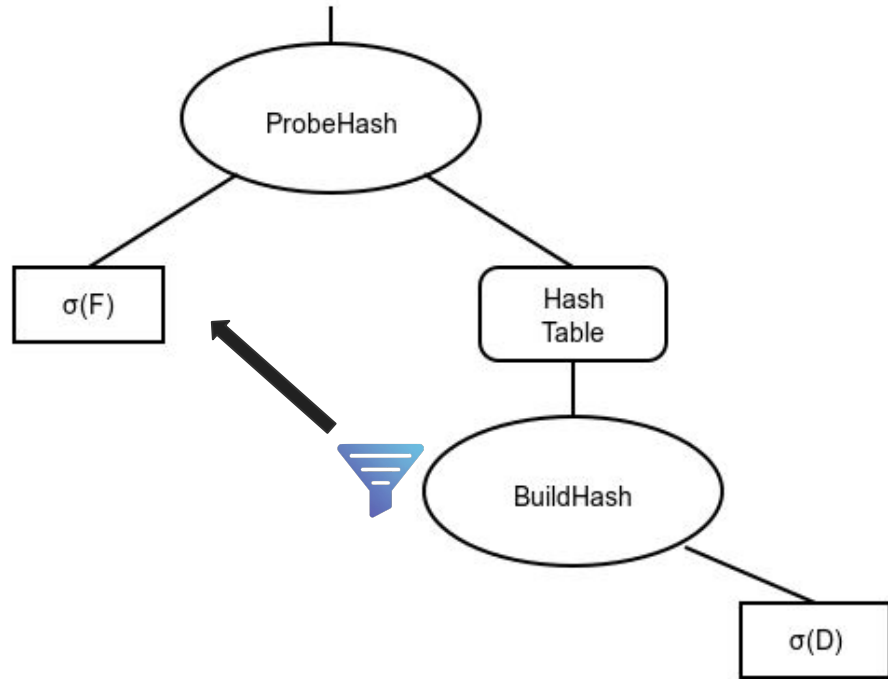
Hash Join Optimization

1. Build a filter along with the hash table



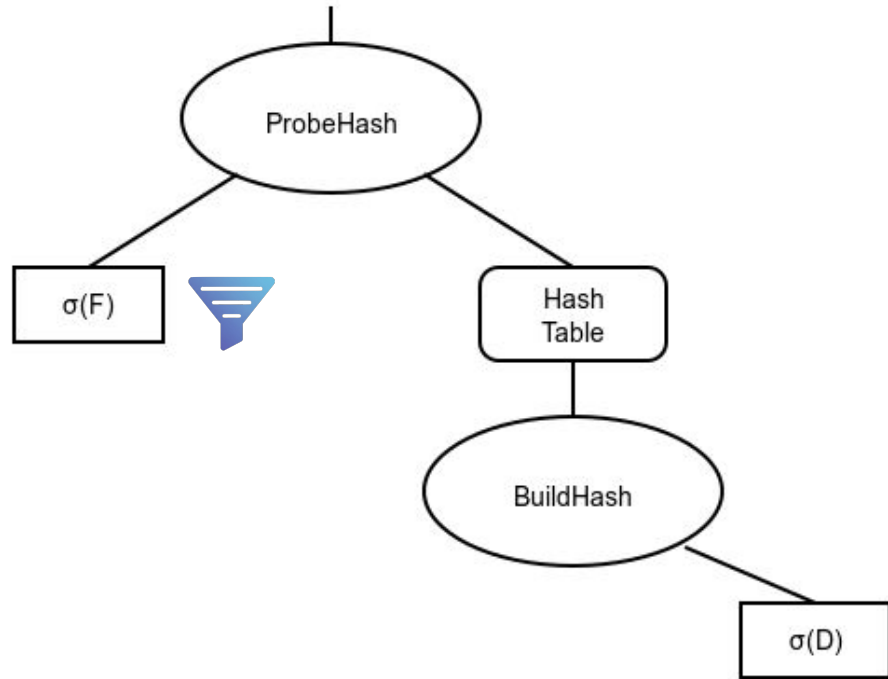
Hash Join Optimization

2 . Pass filter to the probe side.

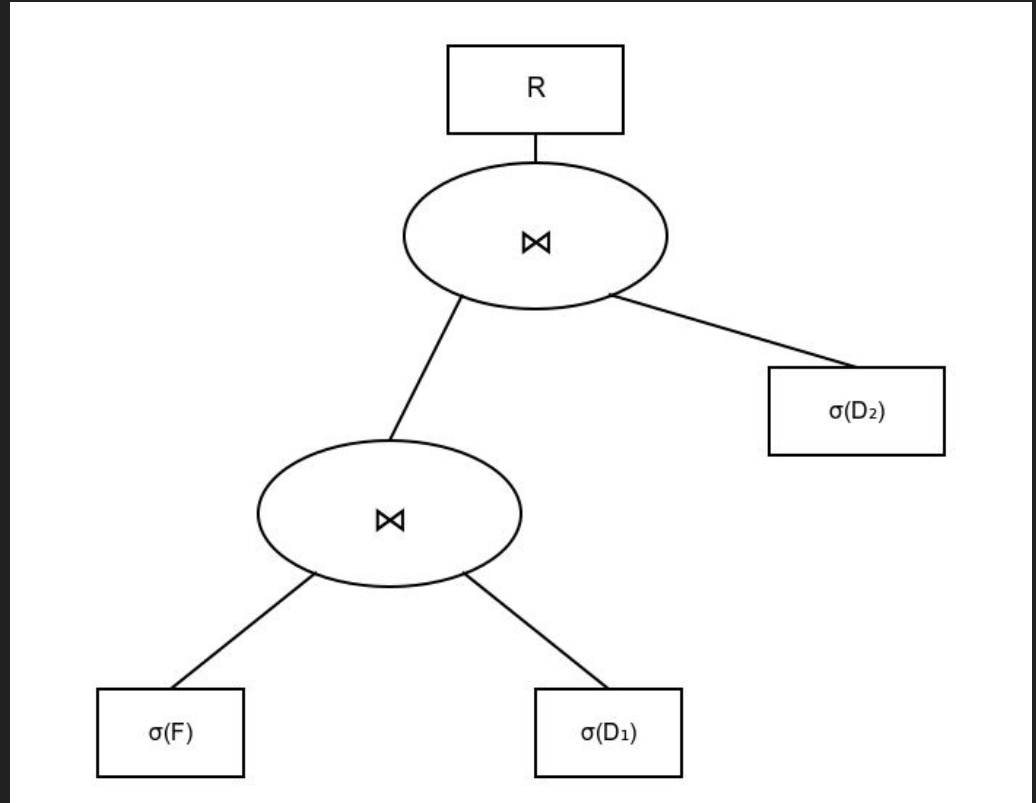


Hash Join Optimization

3. Apply filter at the probe side before probing the hash table.

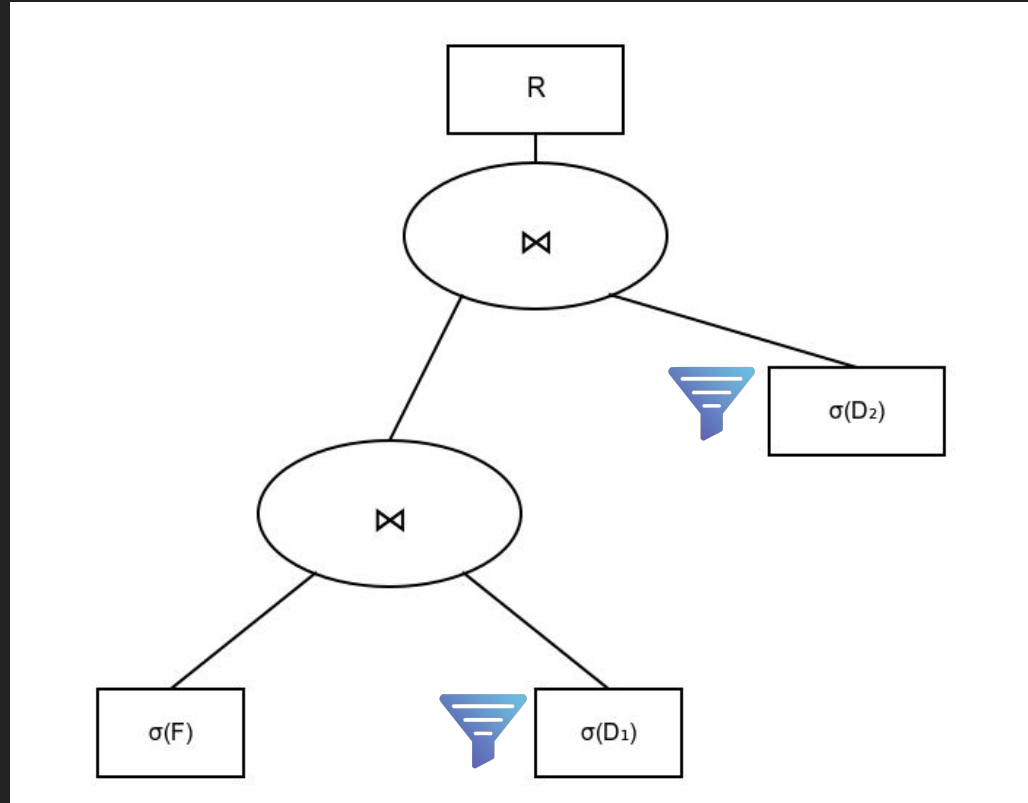


Lookahead Information Passing (LIP)



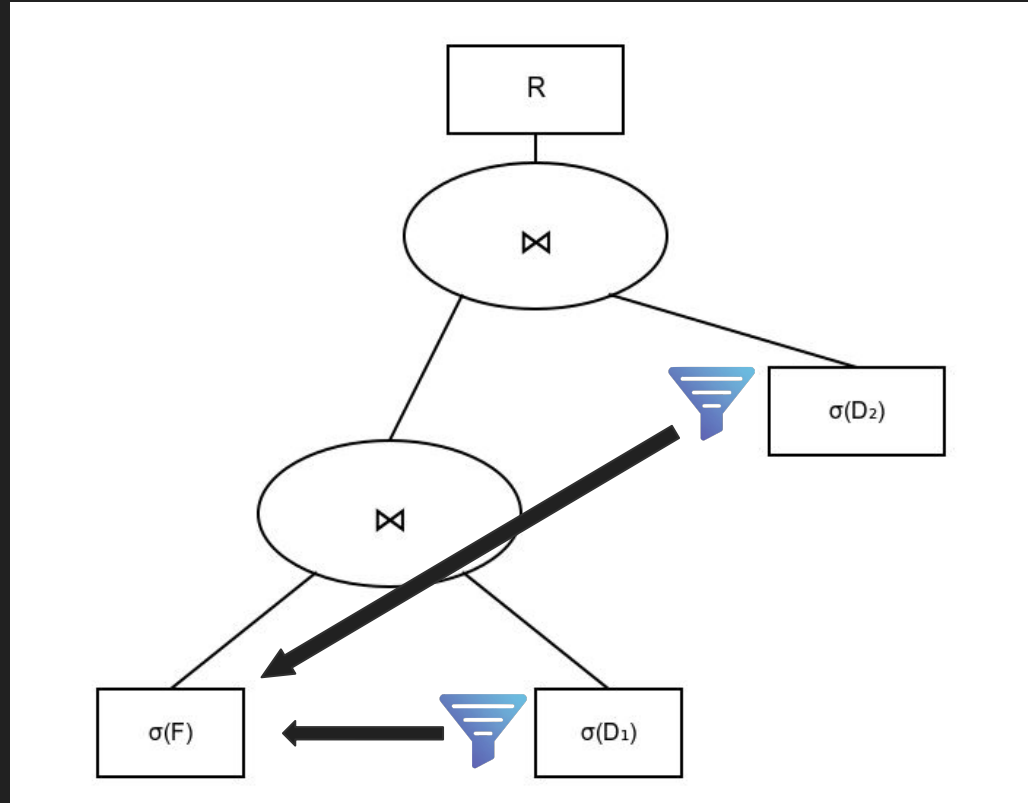
Lookahead Information Passing (LIP)

1. Build a filter along with the hash table



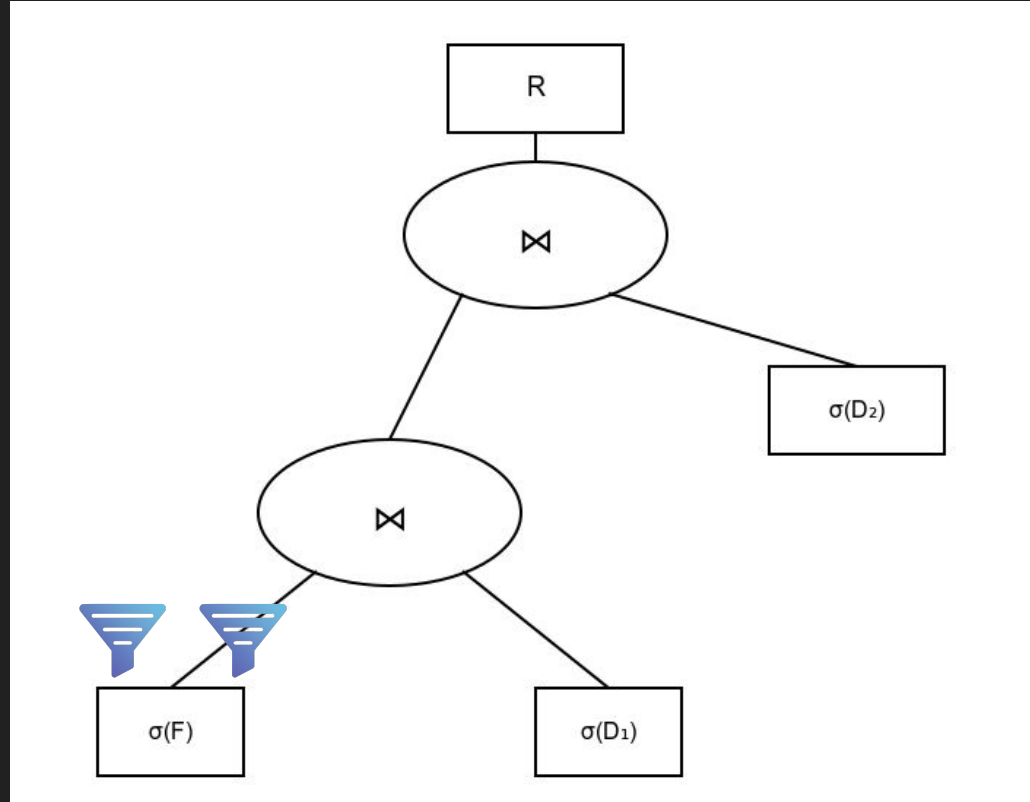
Lookahead Information Passing (LIP)

2 . Pass filter to the probe side.



Lookahead Information Passing (LIP)

3. Apply filter at the probe side before probing the hash table.



Further Optimization

- Adaptive filter ordering
 - Best filter goes first in the next batch
- Bloom filter size fit into cache
 - Avoid the 100x slower penalty for reading RAM

Experiment & Results

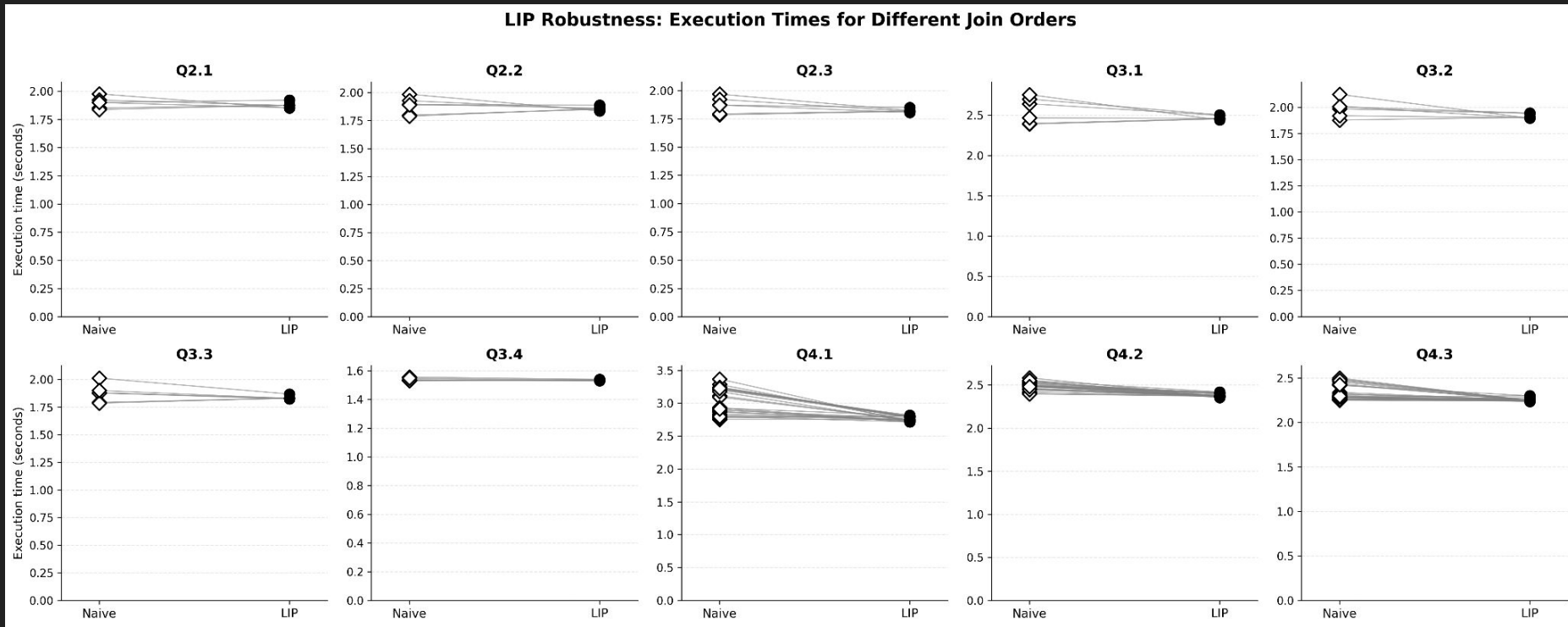
Specifications

- CPU:
 - AMD Ryzen 7 5800X 8-Core Processor
 - Cores: 8 physical cores
 - Max Speed: 3.8 GHz
- RAM: Type: DDR4
 - Size: 32 GB x 4 = 128GB
 - Speed: 2400 MT/s
 - Manufacturer: Kingston
- OS: Ubuntu 22.04.5 LTS

Experiment

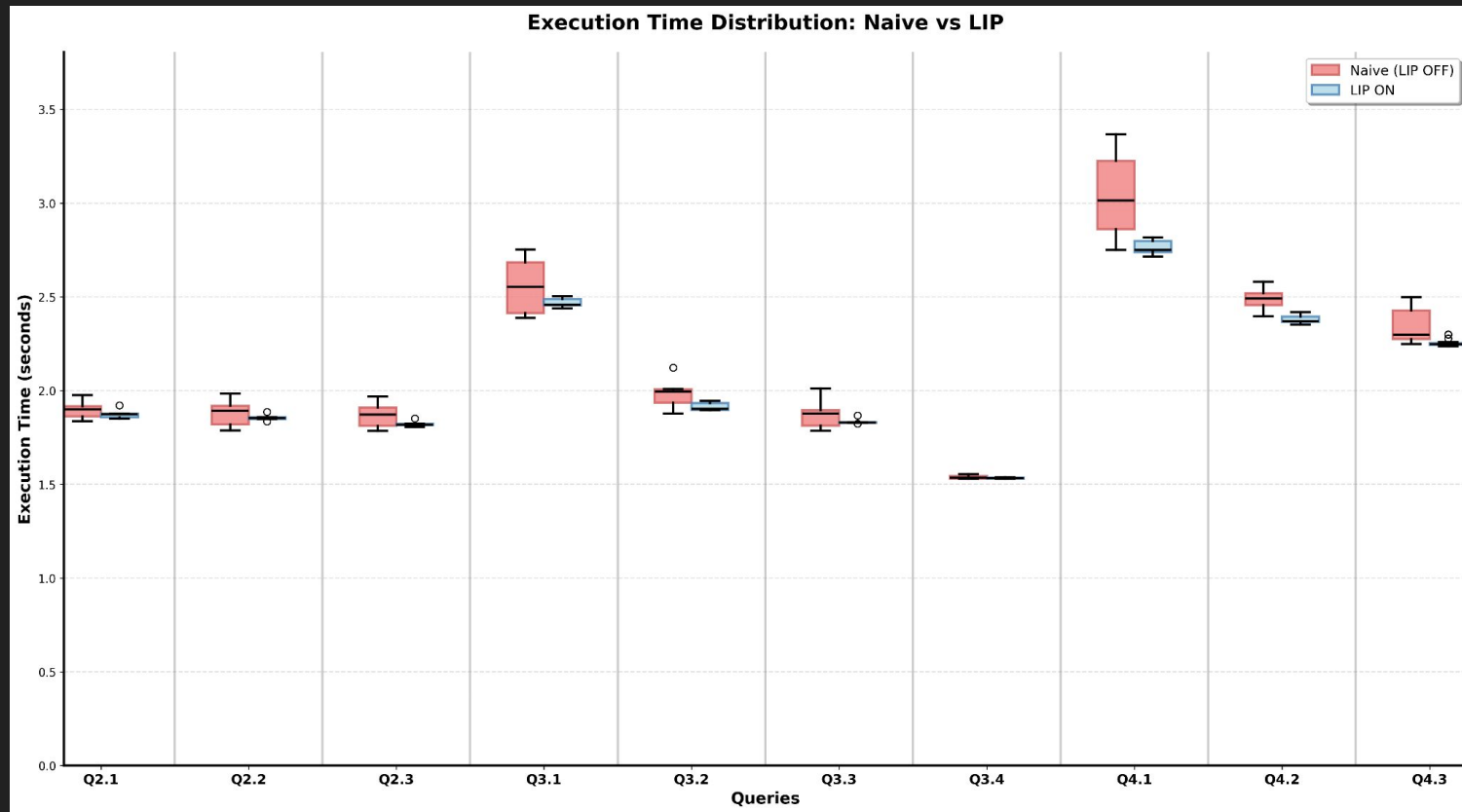
- Run Star Schema Benchmark sf-100
- Run each join order variants 10 times (with LIP on and off)
 - Pick the median
- Before benchmarking, scan all the table (in memory setting)

Execution time of all different join ordering comparison between Naive and LIP



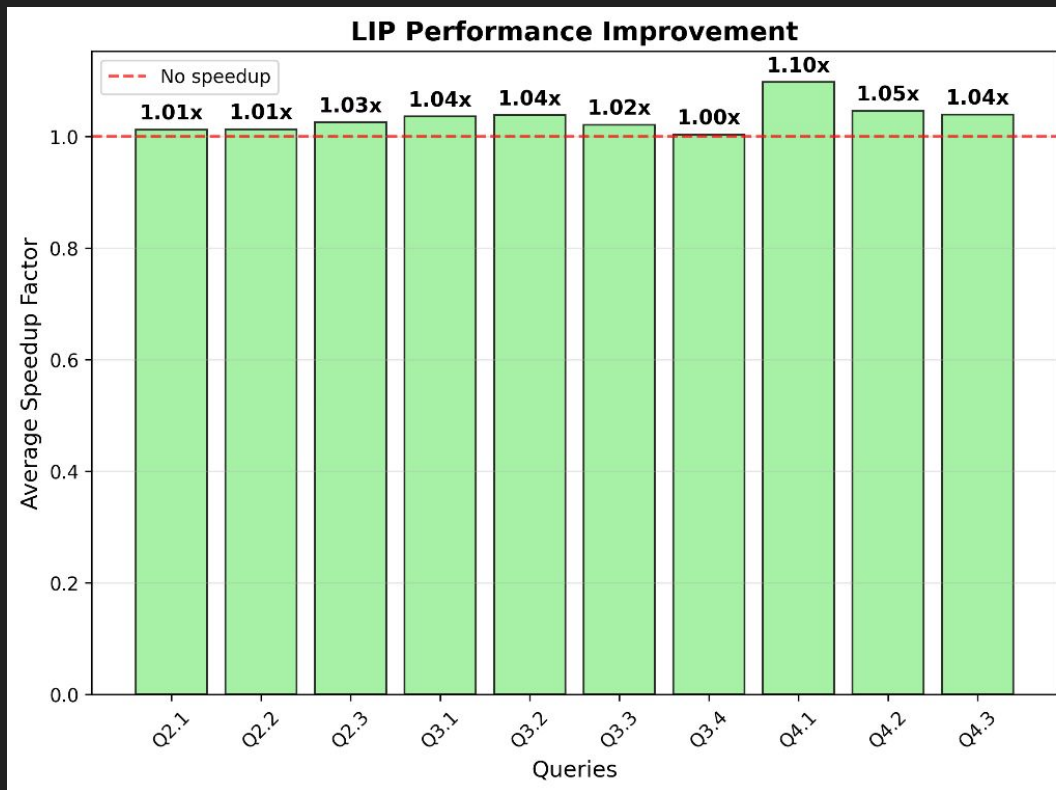
With LIP, query can achieve nearly optimal speed even with poor join order

Execution time distribution



No-LIP query has a wider range of execution time, the queries with LIP are more stable

LIP Speedup



Conclusion

Takeaway

- Star Schema join ordering is common and hard to deal with.
- LIP doesn't just make queries more robust, it makes them faster.
- Doesn't use much memory (filters are small).

Thank you!

Really appreciate your time